

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

**Fabiano Carvalho Costa Júnior – Matrícula: [2520144]
Guilherme Vilas Boas Barbosa – Matrícula: [2411726]**

**Laboratório 02
INF1010**

Rio de Janeiro

2026

1 Objetivo

O objetivo deste trabalho é desenvolver um programa em linguagem C para ler uma sequência de 20 chaves inteiras a partir de um arquivo texto (`entrada.txt`) e inseri-las dinamicamente em uma Árvore Binária de Busca (ABB). A estrutura do nó armazena a chave, a altura e os ponteiros para as subárvores esquerda e direita.

Após a montagem completa da árvore, o programa percorre a estrutura de forma recursiva para calcular e atualizar a altura de cada nó. Por fim, o sistema exibe no console os percursos em pré-ordem e em ordem simétrica, apresentando para cada elemento a sua chave acompanhada da respectiva altura calculada no formato `chave(altura)`, garantindo também a liberação adequada de toda a memória alocada.

2 Estrutura do Programa

- **Estrutura de Dados (Nodo):** Contém os campos inteiros `chave` e `altura`, e dois ponteiros (`esq` e `dir`) para os nós filhos.
- **Módulo de Alocação (`criaNodo`):** Aloca memória dinamicamente com `malloc`, inicializa os ponteiros como `NULL` e define a altura inicial como 0.
- **Módulo de Inserção (`inserir`):** Insere recursivamente cada chave mantendo a propriedade de ordenação da ABB (menores à esquerda, maiores à direita).
- **Módulo de Cálculo de Altura (`calculaAltura`):** Calcula recursivamente a altura de cada nó (definindo folhas com altura 0 e base de cálculo -1 para nós nulos), onde a altura do nó é $1 + \max(\text{altura}_{\text{esq}}, \text{altura}_{\text{dir}})$.
- **Módulos de Percurso (`preOrdem` e `simetrica`):** Percorrem a árvore e imprimem os dados no formato `chave(altura)`.
- **Módulo de Liberação (`liberaArvore`):** Percorre a árvore em pós-ordem desalocando todos os nós com `free`.

3 Solução e Código

O código abaixo realiza a leitura do arquivo de entrada, construção da ABB, cálculo das alturas, exibição e liberação de memória:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct nodo {
5     int chave;
6     int altura;
7     struct nodo *esq;
8     struct nodo *dir;
9 };
10
11 typedef struct nodo Nodo;
```

```

12
13 Nodo *criaNodo(int chave) {
14     Nodo *novo = (Nodo *) malloc(sizeof(Nodo));
15     if (novo == NULL) {
16         printf("Erro ao alocar memoria!\n");
17         exit(1);
18     }
19     novo->chave = chave;
20     novo->altura = 0;
21     novo->esq = NULL;
22     novo->dir = NULL;
23     return novo;
24 }
25
26 Nodo *inserir(Nodo *raiz, int chave) {
27     if (raiz == NULL) {
28         return criaNodo(chave);
29     }
30     if (chave < raiz->chave) {
31         raiz->esq = inserir(raiz->esq, chave);
32     } else if (chave > raiz->chave) {
33         raiz->dir = inserir(raiz->dir, chave);
34     }
35     return raiz;
36 }
37
38 int calculaAltura(Nodo *raiz) {
39     if (raiz == NULL) {
40         return -1;
41     }
42     int alturaEsq = calculaAltura(raiz->esq);
43     int alturaDir = calculaAltura(raiz->dir);
44     if (alturaEsq > alturaDir) {
45         raiz->altura = alturaEsq + 1;
46     } else {
47         raiz->altura = alturaDir + 1;
48     }
49     return raiz->altura;
50 }
51
52 void preOrdem(Nodo *raiz) {
53     if (raiz != NULL) {
54         printf("%d(%d) ", raiz->chave, raiz->altura);
55         preOrdem(raiz->esq);
56         preOrdem(raiz->dir);
57     }
58 }
59
60 void simetrica(Nodo *raiz) {
61     if (raiz != NULL) {
62         simetrica(raiz->esq);

```

```

63         printf("%d(%d)_", raiz->chave, raiz->altura);
64         simetrica(raiz->dir);
65     }
66 }
67
68 void liberaArvore(Nodo *raiz) {
69     if (raiz != NULL) {
70         liberaArvore(raiz->esq);
71         liberaArvore(raiz->dir);
72         free(raiz);
73     }
74 }
75
76 int main(void) {
77     FILE *arquivo;
78     Nodo *raiz = NULL;
79     int chave;
80
81     arquivo = fopen("entrada.txt", "r");
82     if (arquivo == NULL) {
83         printf("Erro_ao_abrir_o_arquivo_entrada.txt\n");
84         return 1;
85     }
86
87     while (fscanf(arquivo, "%d", &chave) == 1) {
88         raiz = inserir(raiz, chave);
89     }
90     fclose(arquivo);
91
92     calculaAltura(raiz);
93
94     printf("Pre-ordem:_");
95     preOrdem(raiz);
96     printf("\n");
97
98     printf("Simetrica:_");
99     simetrica(raiz);
100    printf("\n");
101
102    liberaArvore(raiz);
103    return 0;
104 }

```

3.1 Resultados e Saída do Programa

Utilizando o arquivo `chaves.txt`:

20 30 70 50 100 60 80 90 10 40 5 95 25 85 35 75 65 55 45 41

A execução do programa no terminal gera a seguinte saída:

Pre-ordem: 20(8) 10(1) 5(0) 30(7) 25(0) 70(6) 50(4) 40(2) 35(0)
45(1) 41(0) 60(1) 55(0) 65(0) 100(5) 80(4) 75(0) 90(3) 85(0)
95(2)
Simetrica: 5(0) 10(1) 20(8) 25(0) 30(7) 35(0) 40(2) 41(0) 45(1)
50(4) 55(0) 60(1) 65(0) 70(6) 75(0) 80(4) 85(0) 90(3) 95(2)
100(5)

4 Observações e Conclusões

- **Análise do Resultado:** O percurso em Ordem Simétrica imprime as chaves em ordem crescente, o que confirma visualmente que a estrutura criada é uma Árvore Binária.
- **Cálculo de Altura:** A adoção do valor base -1 para ponteiros `NULL` permitiu que os nós folha (como o nó de chave 5) recebessem corretamente a altura 0, propagando a maior altura do subárvore até a raiz (chave 20, de altura 8).
- **Compilação e Execução:** O programa foi compilado via terminal com o GCC utilizando o comando:

```
gcc -Wall -std=c99 main.c -o arvore  
./arvore
```